

Mind Garden

Final Project Report

AI-Powered Student Wellness Journal

Team Leader: Lamya Faisal Alahmari

Graduation Project | CYB 419 | 2026

Abstract

The Mind Garden Web Application is an AI-powered platform designed to support the mental wellness of university students. Students face growing challenges including academic pressure, financial stress, and social anxiety that negatively impact their psychological well-being and academic performance. This project presents a web-based solution that enables students to write daily emotional journal entries, which are then analyzed using Natural Language Processing (NLP) techniques to identify emotional states such as joy, sadness, anger, fear, and stress. Based on the detected emotion, the system generates personalized wellness recommendations tailored to the individual user. The platform also tracks emotional trends over time and provides weekly and monthly progress reports with visual charts. Built using Python (Flask), SQLite, transformer models, the system addresses key limitations of existing mental health applications by offering culturally sensitive, privacy-focused, and personalized emotional support. This report documents the complete design, implementation, testing, and evaluation of the Mind Garden system.

Acknowledgements

We would like to express our sincere gratitude to our project supervisor, Dr. Manal, for her continuous guidance, support, and encouragement throughout the development of this project. Her expertise and valuable feedback were instrumental in shaping the direction and quality of our work. We also thank the Computer Science and Engineering Department at our university for providing the resources and academic environment that enabled us to pursue this project. Our appreciation extends to all the participants who completed the project questionnaire and provided feedback that helped us understand the needs of the target users. Finally, we thank our families and colleagues for their unwavering support and motivation during the course of this project.

TABLE OF CONTENTS

Abstract.....	II
Acknowledgements	III
1.Introduction.....	1
2.Problem Statement.....	2
3.Background.....	2
4.Requirements and Specifications.....	9
4.1 Functional requirements.	9
4.1.1 User requirements	9
4.1.2 Admin requirements	10
4.2Non-functional Requirements.....	10
4.3 Technical Specifications:.....	11
5.System Design	12
5.1.Solution Concept	12
5.2 Project Contribution	13
5.3Architecture	14
5.2.Component Design	17
5.3.1 Use Case Model:.....	17
5.3.2 Description of Use Case:	18
5.4 System Integration	23
5.4.1 Sequence Diagram:	23
5.4.2 ER diagram:	24
5.5 Design Evaluation	26
5.5.1 User interfaces.....	26
5.5.2 Admin interfaces.....	31
6.Testing, Analysis and Evaluation.....	32
6.1 Testing.....	32
1. Functional Testing.....	32
2. Unit Testing.....	32
6.2 Analysis	33
6.3 Evaluation	33

7.Issues	34
8.Engineering Tools and Standards	35
9.Teamwork	36
10.Progress.....	36
11. Conclusions	38
11.1 Conclusions	38
11.2 Future Work.....	39
Reference.....	40
Appendix	41
B. Source Code	43
B.1 requirements.txt.....	43
B.2 run.py – Application Entry Point.....	43
B.3 config.py – Application Configuration	43
B.4 app/__init__.py – Application Factory	44
B.5 app/auth/routes.py – Authentication Routes	44
B.6 app/main/routes.py – Main Application Routes	46
B.7 app/admin/routes.py – Admin Panel Routes	48
B.8 app/utls/nlp_service.py – NLP Emotion Analysis Service.....	50
B.9 app/utls/text_processing.py – Text Processing Utilities	51

LIST OF FIGURES

Figure 1: Chart of the responses received – Age distribution.....	4
Figure 2 Chart of the responses received – Daily device usage.....	4
Figure 3 Chart of the responses received – Academic pressure during the semester	5
Figure 4: Chart of the responses received – Causes of stress	5
Figure 5 Chart of the responses received – Coping strategies under psychological pressure....	6
Figure 6 Chart of the responses received – Use of mental health or mood tracking	6
Figure 7 Chart of the responses received – Reasons for hesitating to visit a psychologist.....	7
Figure 8 Interest in a mental health application for daily emotional journaling	7
Figure 9 Chart of the responses received – Preferences for tailored psychological advice	8
Figure 10 Chart of the responses received	8
Figure 11: Perceived usefulness of a mental health application	9
Figure 12 Architecture	15
Figure 13 Use Case Model.....	17
Figure 14 Sequence Diagram:	23
Figure 15 ER diagram:	24
Figure 16 Home Page – Landing Screen.....	26
Figure 17 Mood History Page – Journal Records Table	27
Figure 18 Emotional Reports Page – Monthly Report (Light Mode)	28
Figure 19 Dashboard – Daily Emotional Journal with NLP Analysis Result.....	28
Figure 20 Emotional Reports Page – Weekly Report (Dark Mode)	29
Figure 21 Login Page – User Authentication Screen.....	30
Figure 22 Register Page – New Account Creation Screen	30
Figure 23 User Management Dashboard.....	31
Figure 24 Recommendation Rules Panel	31
Figure 25 Project questionnaire	42

LIST OF TABLES

Table 1 Technical Specifications	11
Table 2 Hardware Requirements.....	12
Table 3 Use Case 1: Register Account.....	18
Table 4 Use Case 2: Login	19
Table 5 Use Case 3: Write Emotional Post.....	19
Table 6 Use Case 4: Emotion Analysis.....	20
Table 7 Use Case 5: Generate Recommendation	20
Table 8 Use Case 6: View Mood History	21
Table 9 Use Case 7: Generate Weekly/Monthly Report.....	21
Table 10 Use Case 8: Manage Users (Admin).....	22
Table 11 Use Case 9: Manage Recommendations (Admin)	22
Table 12 Issues	35
Table 13 Teamwork	36
Table 14 Phase 1: Requirements	37
Table 15 Phase 2: System Design	37
Table 16 Phase 3: Implementation	38

1. Introduction

Academic pressure, social expectations, financial issues, and future uncertainty have been the major causes of mental health issues among university students over the past few years. Anxiety and stress disorder are some of the most prevalent psychological problems in young adults according to the World Health Organization (WHO) [7]. Research indicates that a high proportion of college students have moderate to high stress levels in the course of studying.

Mental health has turned out to be a burning problem due to the direct impact it has on:

1. Academic performance
2. Productivity
3. Social relationships
4. Overall quality of life

Stress and anxiety not treated can cause severe effects like burnout, depression or quitting academic life. Even as the students require more and more help, there is a stigma, cost, or even access to mental health services, which is why many students are reluctant to resort to professional assistance.

The proposed solution to this dilemma is the Mind Garden Web application, an AI-driven web-application, where students could share their feelings by writing, their emotional state would be analyzed through the application of Natural Language Processing (NLP), and some wellness tips would be given. The platform also keeps a record of their emotional trends with time and creates progress reports so that users can have a clearer explanation of their mental health. The system provides aids of preventive and accessible mental wellness support with the help of expressive writing, artificial intelligence, and personal advisory services.[1]

2. Problem Statement

Students in the university are getting more exposed to academic pressure, stress and emotional challenges which adversely impacts their mental health and performance in school. The stigma, cost or inability to get easily available services make many students have a hard time expressing their feelings or getting professional psychological help.

The majority of the available mental wellness apps:

- Give non-personalized and general recommendations.
- Deny the users the ability to monitor and learn about their emotional patterns with time.

Thus, a smart, culturally adjusted web application that allows university students to share their emotion safely, analyzes their mental condition with the help of Artificial Intelligence, and offers individual mental health assistance without violating their privacy and ease of use is required.

3. Background

3.1 Related Terminology and Concepts.

- Mental Health: The emotional, psychological and social wellbeing of a person.
- Expressive Writing: This is a type of therapy that enables a person to use writing to vent out his/her feelings to alleviate stress.
- Artificial Intelligence (AI): This refers to computer systems which can execute tasks which involve human intelligence.
- Natural Language Processing (NLP): This is a subdivision of AI that allows machines to comprehend and analyze human speech.[2]
- Emotion Analysis: Recognizing emotions on textual data.

- **Personalized Recommendation Systems:** This is the one that offers customized recommendations, based on user actions and preferences.
- **Mood Tracking:** Tracking emotional conditions and tracking them through time to see patterns and trends.

3.2 Related Technologies

- **web Development** - web application in python.[3]
- **AI/ NLP:** NLP models of emotion classification
- **API Services:** Text analysis and recommendations processing APIs.
- **Databases:** Mood history and recommendations of users.
- **Data Protection:** To prevent any breach of confidential psychological information, encryption and secure authentication.

3.3 Existing Solutions

Current mental welfare applications like meditation or mood tracking apps offer general functionality like breath exercises or reminders. However, these solutions:

- Primarily targeted at English speakers.
- Give impersonal recommendations.
- Give minimal emotional intelligence in terms of reports.

3.4 The Questionnaire

A questionnaire is an important tool and it is common knowledge that when studying a phenomenon or problem, it should collect information and data; To clarify and analyze the nature of the phenomenon or problem to be pursued in the research. When the target users have responded to the questionnaire, we collected the statistics of the results as shown in the following charts:

الفئة العمرية

Age Group

143 ردًا

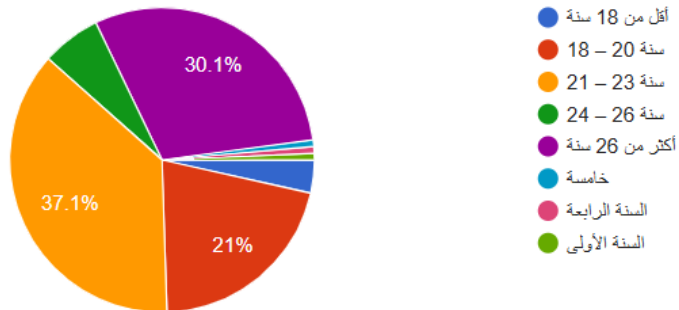


Figure 1: Chart of the responses received – Age distribution

كم ساعة تفضي يوميًا على الأجهزة (موبايل/كمبيوتر)؟

How many hours do you spend daily on devices (mobile/computer)

143 ردًا

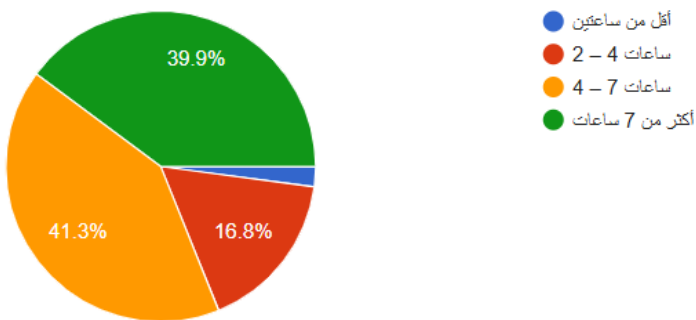


Figure 2 Chart of the responses received – Daily device usage

هل تشعر بضغوط دراسي خلال الفصل الدراسي؟
Do you feel academic stress during the semester
 141 ردًا

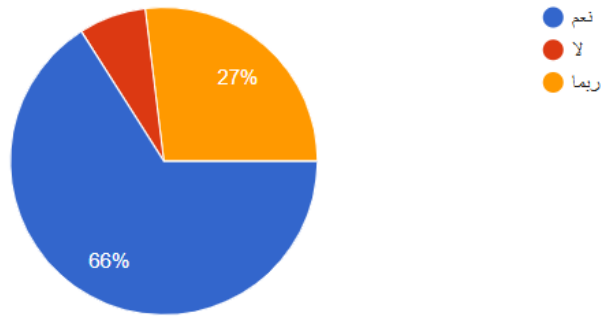


Figure 3 Chart of the responses received – Academic pressure during the semester

ما أكثر سبب يسبب لك التوتر؟
What is the main cause of your stress
 143 ردًا

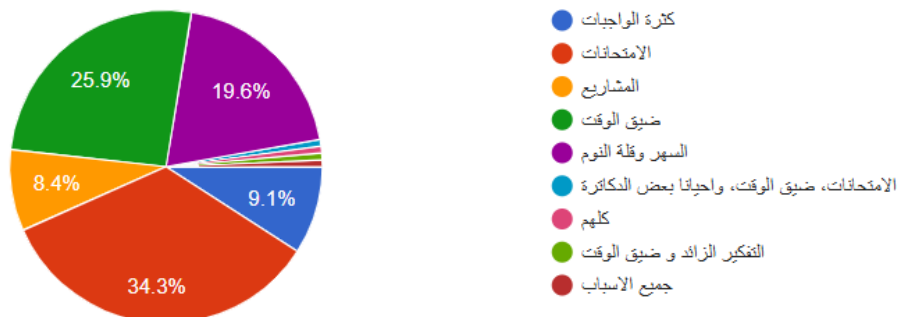


Figure 4: Chart of the responses received – Causes of stress

عندما تشعر بالضغط النفسي ماذا تفعل عادة؟

What do you usually do when you feel psychological stress

142 ردًا

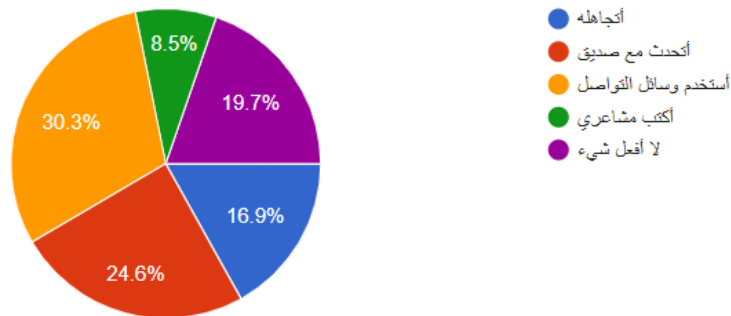


Figure 5 Chart of the responses received – Coping strategies under psychological pressure

هل سبق لك استخدام تطبيق للصحة النفسية أو تتبع المزاج؟

Have you ever used a mental health or mood tracking app

142 ردًا

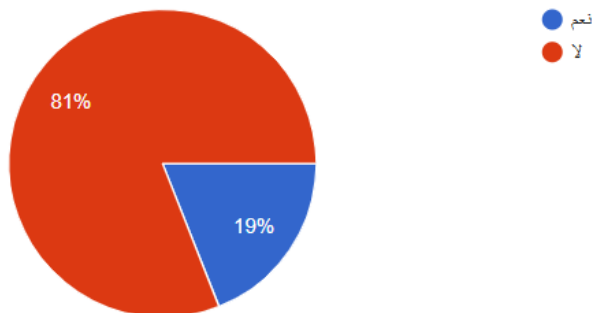


Figure 6 Chart of the responses received – Use of mental health or mood tracking

لماذا قد تتردد في الذهاب إلى معالج نفسي؟
Why might you hesitate to visit a therapist

141 ردًا

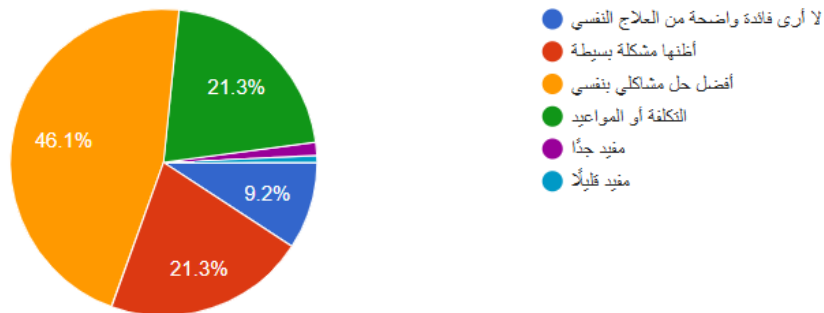


Figure 7 Chart of the responses received – Reasons for hesitating to visit a psychologist

هل تحب وجود تطبيق تستطيع فيه كتابة مشاعرك يوميًا ويحلل حالتك النفسية تلقائيًا؟
Would you like an app where you can write your feelings daily and have your
mental state analyzed automatically

143 ردًا

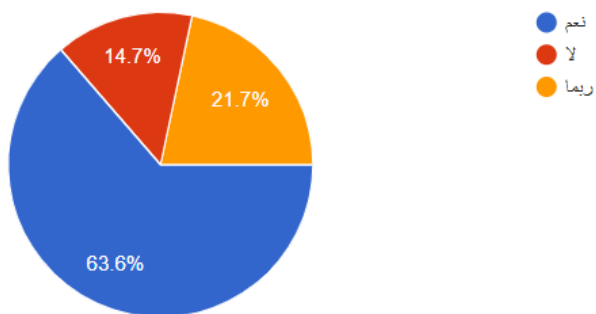


Figure 8 Interest in a mental health application for daily emotional journaling and automatic psychological analysis

هل تفضل أن تكون النصائح مخصصة لك بناءً على حالتك النفسية؟
Do you prefer personalized advice based on your mental state

143 ردًا

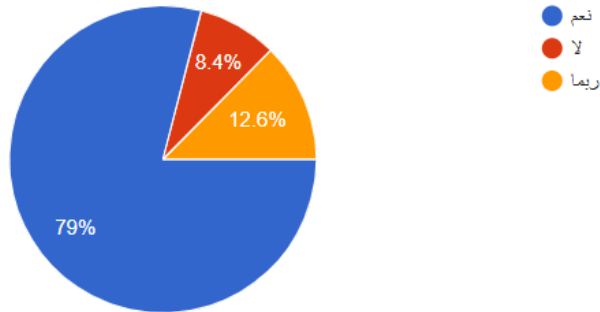


Figure 9 Chart of the responses received – Preferences for tailored psychological advice

هل تشعر بالراحة في استخدام تطبيق مجهول الهوية (لا يظهر اسمك) لمشاركة مشاعرك؟
Do you feel comfortable using an anonymous app (that does not show your name) to share your feelings

142 ردًا

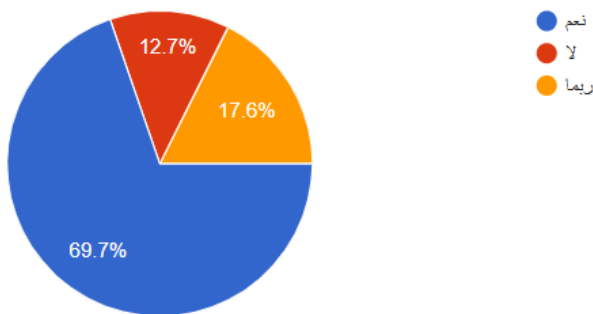


Figure 10 Chart of the responses received – Comfort with using an anonymous application to share feelings

هل تعتقد أن تطبيق كهذا قد يساعد في تحسين حالتك النفسية أو تنظيم دراستك؟
Do you think such an app could help improve your mental state or organize your studies

143 ردًا

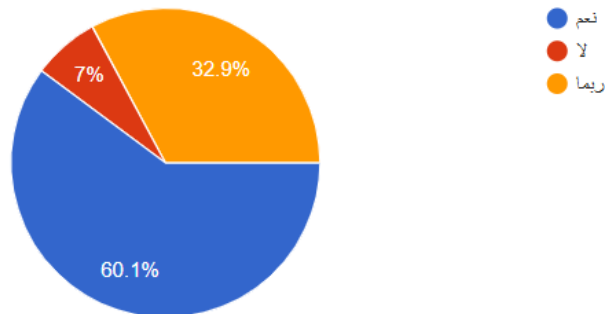


Figure 11: Perceived usefulness of a mental health application for improving psychological state

4. Requirements and Specifications

4.1 Functional requirements.

4.1.1 User requirements

The system shall allow university students (users) to perform the following functions:

4.1.1.1. Account Management

- The system shall allow users to register using email and password.
- The system shall allow users to securely log in and log out.
- The system shall allow users to edit their profile information (name, preferences, etc.).

4.1.1.2. Emotional Expression

- The system shall allow users to write daily emotional posts in text form.
- The system shall allow users to include emojis in their posts.

4.1.1.3. Emotion Analysis

- The system shall analyze user-written text using NLP techniques.
- The system shall classify emotions into:

- ✓ Stress
- ✓ Anxiety
- ✓ Sadness
- ✓ Calmness
- ✓ Positivity
- ✓ Motivation

- The system shall display the detected emotion to the user within 3–5 seconds.

4.1.1.4. Personalized Recommendations

- The system shall provide personalized wellness recommendations based on the detected emotion.
- The system shall display recommendations immediately after emotion analysis.

4.1.1.5. Mood Tracking & Reports

- The system shall store emotional history securely in the database.
- The system shall allow users to view their mood history.
- The system shall generate weekly progress reports.
- The system shall generate monthly progress reports.
- The system shall visualize emotional trends using graphs and charts.

4.1.2 Admin requirements

4.2 Non-functional Requirements

4.2.1 Performance:

The system shall analyze text and return results within **3–5 seconds**.

4.2.2 Security:

User data shall be encrypted and securely stored.

Authentication shall use secure mechanisms.

4.2.3 Usability:

The application shall be easy to use and suitable for non-technical users.

4.2.4 Privacy:

Psychological data shall be protected and accessed only by authorized users.

4.2.5 Compatibility:

The application shall run on modern web browsers (Chrome, Edge, Firefox) across desktop and mobile devices.

4.2.6 Scalability:

The system shall support future extensions such as voice-to-text and wearable integration.

4.2.7 Cost:

The system shall rely on open-source tools to minimize development and deployment cost.

4.3 Technical Specifications:

Table 1 Technical Specifications

Specification	Description
Platform	web app Application
Programming Language	python
Architecture	Client–Server
AI Component	NLP-based emotion analysis
Input	User-written text and emojis

Output	Emotion label + personalized recommendations
Recommendation Type	Content-based personalized system
Database	Secure relational
Security	Authentication, encryption, secure APIs
Testability	Response time, emotion classification accuracy, system reliability

Table 2 Hardware Requirements

Hardware Component		Minimum Specification
Laptop / PC		Intel Core i5 or equivalent
RAM		8 GB
Storage		256 GB SSD
Internet Connection		Stable broadband connection
Mobile Device (Optional)		Android / iOS smartphone
Browser		Latest version of Chrome / Edge / Firefox

5. System Design

This section describes the overall technical design of the **Mind Garden** web application, including the solution concept, algorithms used, system architecture, and component structure.

5.1.Solution Concept

In addition to the proposed functionality, it is important to compare the Mind Garden system with existing mental health applications to highlight its unique contribution.

Several existing platforms such as mood tracking and meditation applications provide basic features like breathing exercises, reminders, or general emotional logging. However, these systems often suffer from the following limitations:

- They provide generic (non-personalized) recommendations
- They lack deep emotion analysis using AI techniques
- They do not offer long-term emotional pattern tracking with intelligent insights
- Many are not culturally adapted for Arabic-speaking users

The Mind Garden system addresses these limitations by introducing the following key contributions:

- AI-powered emotion analysis using NLP, enabling accurate detection of user emotions from text
- Personalized recommendations tailored to each user's emotional state
- Continuous mood tracking with visual reports (weekly/monthly insights)
- Anonymous usage, encouraging users to express feelings without fear or stigma

Therefore, Mind Garden is not just a tracking tool, but an intelligent emotional support system that combines accessibility, personalization, and privacy.

5.2 Project Contribution

The main contribution of this project is the development of an intelligent, user-centered mental health support system specifically designed for university students. Unlike traditional mental health applications that provide general and non-personalized support, this system focuses on delivering a more adaptive and meaningful experience through the integration of artificial intelligence techniques.

The system utilizes Natural Language Processing (NLP) to analyze the emotional content of users' written text and automatically identify their psychological state. Based

on this analysis, it generates personalized wellness recommendations that are tailored to the user's current emotional condition rather than offering generic advice.

In addition, the platform emphasizes privacy and user comfort by allowing anonymous interaction, which helps reduce the stigma associated with discussing mental health issues. This encourages users to express their feelings more openly and honestly.

Another important contribution of the project is the ability to track emotional patterns over time. The system stores user entries and provides visual reports that help users understand their mood trends and emotional changes across different periods, such as weekly and monthly summaries.

Overall, the project is specifically designed to address the psychological challenges faced by university students, particularly academic stress and emotional pressure, by combining AI-driven analysis, personalization, and privacy-focused design into a single integrated platform.

5.3Architecture

This section describes the overall system architecture of the **Mind Garden** web application, including architectural style, components, alternatives, and the functions of each part.

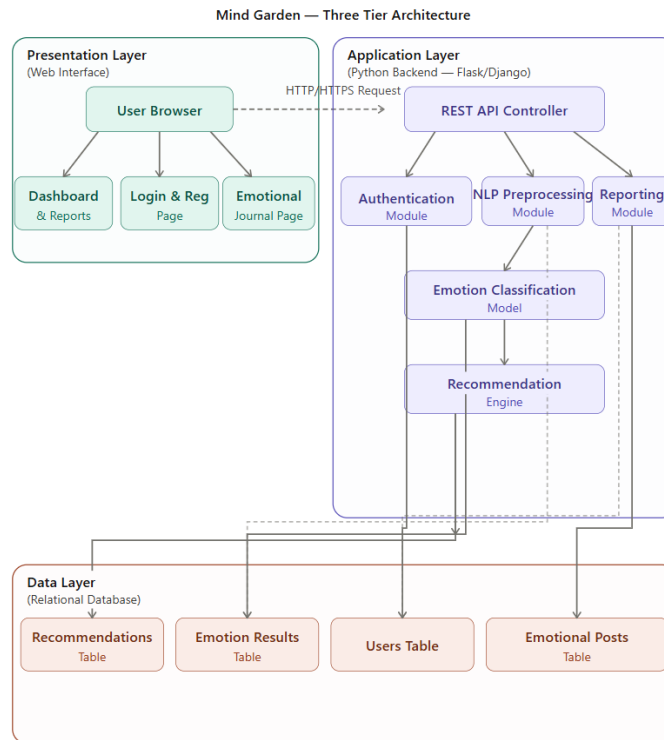


Figure 12 Architecture

1. Presentation Layer (Frontend)

Technologies:

- HTML/CSS
- JavaScript
- Bootstrap

Functions:

- User registration and login interface
- Emotional journal input form
- Display detected emotion
- Show personalized recommendations
- Display weekly and monthly reports
- Visualize mood trends using charts

2. Application Layer (Backend)

Technology:

- Python (Flask or Django framework)

Functions:

- Handle authentication and authorization
- Process user requests
- Execute NLP preprocessing
- Perform emotion classification
- Generate personalized recommendations
- Store and retrieve data from database
- Enforce security rules
- Provide REST API endpoints

3. Data Layer (Database)

Type:

- Relational Database (SQL Lite)

Stores:

- User account information
- Encrypted passwords
- Emotional journal entries
- Predicted emotion labels
- Recommendation history
- User feedback data

5.2.Component Design

5.3.1 Use Case Model:

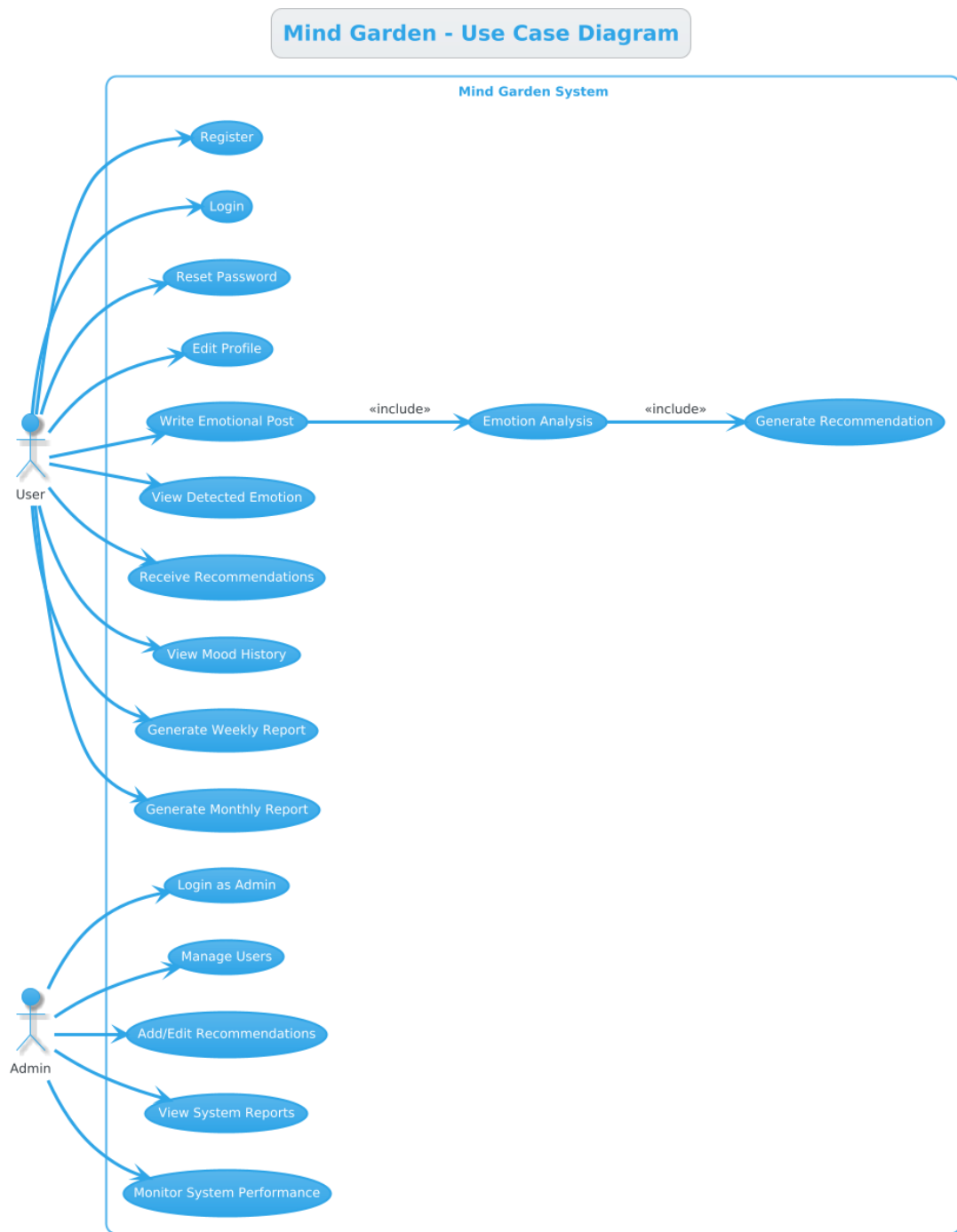


Figure 13 Use Case Model

5.3.2 Description of Use Case:

In this section, we will describe each use case by identifying its Name, Actors, Description, Preconditions, Postconditions, and Base Use cases.

- A. Name: Give a short, descriptive name to the use case
- B. Actors: List the actors who can perform this use case
- C. Description: Explain what the actor or actors are trying to achieve
- D. Preconditions: state that the system must be in for the use case to be able to start.
- E. postconditions: lists possible states that the system can be in after the use case runs.
- E. Base use case: includes its functionality to another use case.

Use Case 1: Register Account

Table 3 Use Case 1: Register Account

Field	Description
Name	Register Account
Actors	User
Description	The user creates a new account by providing required personal information such as email and password.
Preconditions	User is not logged into the system.
Postconditions	A new user account is created and stored in the database.
Base Use Case	None

Use Case 2: Login

Table 4 Use Case 2: Login

Field	Description
Name	Login
Actors	User, Admin
Description	The actor enters credentials to access the system dashboard.
Preconditions	Actor must have a registered account.
Postconditions	Actor is authenticated and redirected to dashboard.
Base Use Case	None

Use Case 3: Write Emotional Post

Table 5 Use Case 3: Write Emotional Post

Field	Description
Name	Write Emotional Post
Actors	User
Description	The user writes a daily emotional journal entry in text form, optionally including emojis.
Preconditions	User must be logged into the system.
Postconditions	Emotional text is stored in the database and sent for analysis.
Base Use Case	Includes: Emotion Analysis

Use Case 4: Emotion Analysis

Table 6 Use Case 4: Emotion Analysis

Field	Description
Name	Emotion Analysis
Actors	System
Description	The system processes and analyzes the submitted text using NLP techniques to detect the user's emotional state.
Preconditions	Emotional post must be submitted.
Postconditions	Emotion label is generated and stored in database.
Base Use Case	Included by: Write Emotional Post

Use Case 5: Generate Recommendation

Table 7 Use Case 5: Generate Recommendation

Field	Description
Name	Generate Recommendation
Actors	System
Description	The system generates personalized wellness recommendations based on detected emotion.
Preconditions	Emotion must be successfully detected.
Postconditions	Recommendation is displayed and stored.
Base Use Case	Included by: Emotion Analysis

Use Case 6: View Mood History

Table 8 Use Case 6: View Mood History

Field	Description
Name	View Mood History
Actors	User
Description	The user views previous emotional records and trends.
Preconditions	User must be logged in and have existing records.
Postconditions	Mood history and visual charts are displayed.
Base Use Case	Uses stored emotional data

Use Case 7: Generate Weekly/Monthly Report

Table 9 Use Case 7: Generate Weekly/Monthly Report

Field	Description
Name	Generate Progress Report
Actors	User
Description	The user generates a weekly or monthly emotional progress report.
Preconditions	Sufficient historical emotional data exists.
Postconditions	Report is generated and displayed in graphical format.
Base Use Case	Includes: View Mood History

Use Case 8: Manage Users (Admin)

Table 10 Use Case 8: Manage Users (Admin)

Field	Description
Name	Manage Users
Actors	Admin
Description	The admin views, edits, suspends, or deletes user accounts.
Preconditions	Admin must be logged into the admin panel.
Postconditions	User account status is updated in the database.
Base Use Case	Requires: Admin Login

Use Case 9: Manage Recommendations (Admin)

Table 11 Use Case 9: Manage Recommendations (Admin)

Field	Description
Name	Manage Recommendations
Actors	Admin
Description	The admin adds, edits, or deletes wellness recommendations.
Preconditions	Admin must be logged in.
Postconditions	Recommendation database is updated.
Base Use Case	Requires: Admin Login

5.4 System Integration

5.4.1 Sequence Diagram:

A sequence diagram is a type of interaction diagram, it describes how and in what order a group of objects works together. Sequence diagrams are used by software developers and business professionals to understand requirements for a new system or to document an existing process. It is sometimes known as event diagrams or event scenarios.[3]

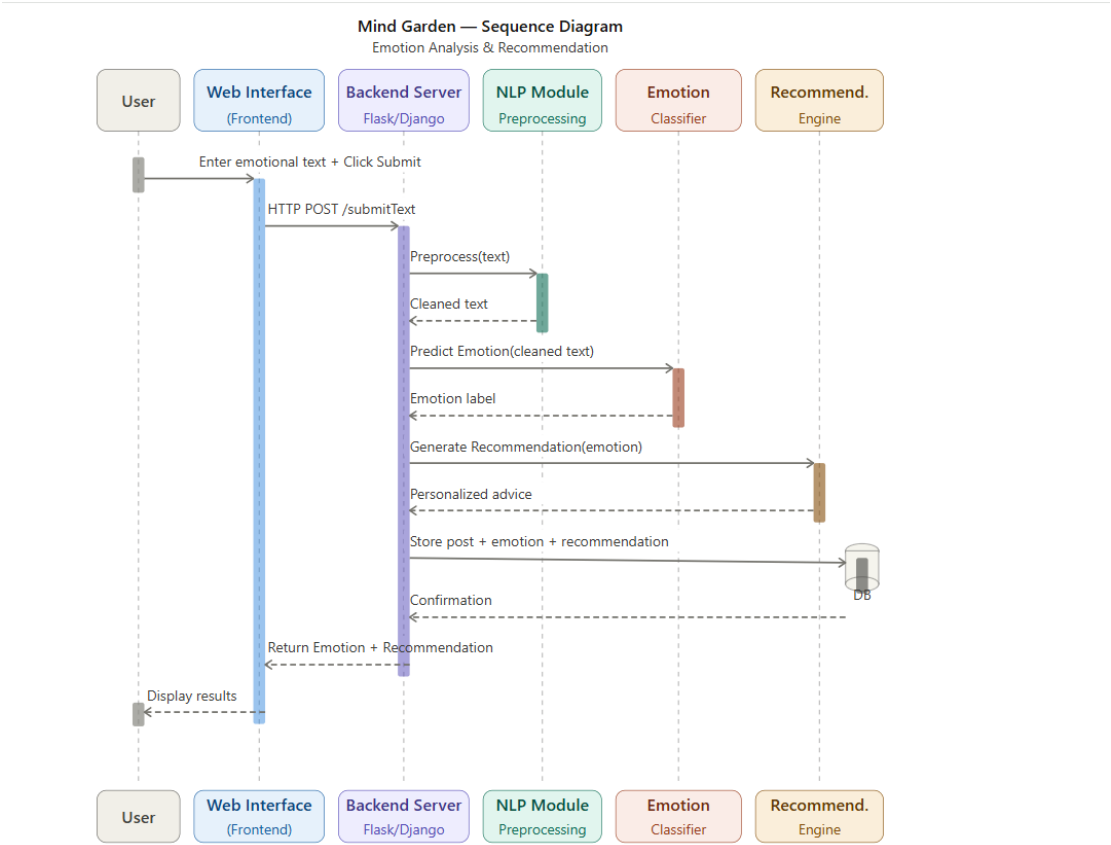


Figure 14 Sequence Diagram:

5.4.2 ER diagram:

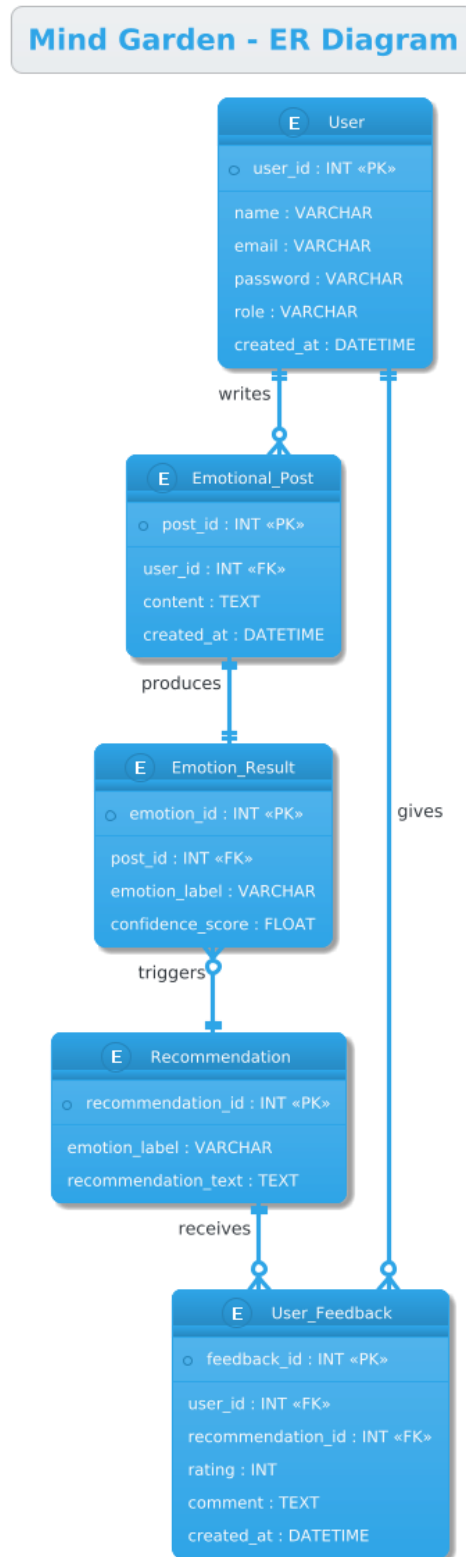


Figure 15 ER diagram:

As shown in the figure above, there are many important relationships, and they will be detailed:

Entity Relationship (ER) diagram of the Mind Garden system is the logical database design and shows how various entities communicate with each other in the system. The database is modeled in a relational model so as to provide data integrity, consistency, and effective retrieval of the psychological records.

The key players within the system are:

- **User**

User entity is where account details of all registered users are stored. The user (Primary Key) is a unique user-id. Others are name, email, password, role and date of account creation.

One user is able to make numerous emotional posts and give comments on suggestions.

- **Emotional_Post**

The Emotional_Post object contains entries of journal on a daily basis that users post. The emotional text and the timestamp are stored in each post which has a post id (Primary Key).

The user_id is a Foreign Key, which connects every post to the user who posted the post.

The relationship between User and Emotional_Post is one to many i.e. there are many posts that can be created by one user.

- **Emotion_Result**

The Emotion Result entity contains the emotion found on each emotional post. It contains a distinct emotion id (Primary key) the label of the emotion predicted and a confidence score.

An emotional post generates only one emotional response hence the relationship between Emotional_Post and Emotion_Result is a one to one relationship.

- **Recommendation**

The Recommendation body keeps wellness information with regards to various emotional conditions. The recommendations contain the emotion category and the text of the recommendation as well as the recommendation_id (Primary Key).

An emotion that is detected initiates a recommendation. Different users can be provided with the same recommendation due to the similarity in the perceived emotions.

5.5 Design Evaluation

5.5.1 User interfaces

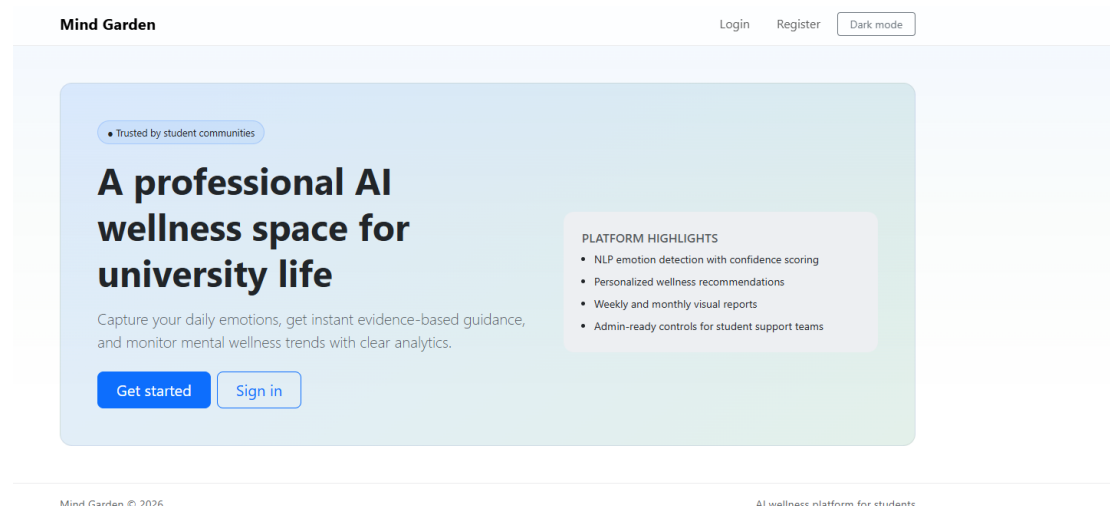


Figure 16 Home Page – Landing Screen

The Home Page of the Mind Garden app is a welcoming entry point designed for university students. It features a clean hero section with the tagline “A professional AI wellness space for university life” and encourages users to log their emotions, receive AI-based guidance, and track mental health trends. It highlights key features such as emotion detection using NLP, personalized wellness tips, visual progress reports, and admin support tools. The page includes “Get started” and “Sign in” buttons for easy access, along with a navigation bar offering Login, Register, and a Dark Mode option.

Mind Garden

Dashboard
History
Reports
Admin
Logout

Dark mode

Mood History

Review your complete emotional journaling timeline.

JOURNAL RECORDS

DATE	ENTRY	EMOTION	CONFIDENCE	KEYWORDS	ANONYMOUS
2026-03-03 13:57	fake_student_journals	anger	0.58	fake_student_journals	No
2026-03-03 13:52	I'm overwhelmed with deadlines, but I'm still hopeful I can manage everything if I plan better.	surprise	0.93	overwhelmed deadlines, deadlines hopeful, deadlines, manage plan, hopeful manage	No
2026-03-03 13:51	I feel sad and low energy today after an argument with my close friend.	sadness	0.99	feel sad, sad low, close friend, low energy, argument close	No
2026-03-03 13:51	I had a normal day, attended classes, studied a bit, and just relaxed in the evening.	joy	0.82	attended classes, relaxed evening, classes studied, day attended, relaxed	No
2026-03-03 13:51	Today was amazing — I finished my project presentation and felt really proud and excited!	joy	0.97	project presentation, presentation felt, proud excited, presentation, finished project	No
2026-03-03 13:51	Today was amazing — I finished my project presentation and felt really proud and excited!	joy	0.97	project presentation, presentation felt, proud excited, presentation, finished project	No
2026-03-03 13:50	I feel stressed and anxious about my exams next week, and I'm not sleeping well.	fear	0.71	anxious exams, stressed anxious, feel stressed, stressed, exams week	No
2026-03-03 13:50	Today I'm excited and happy after finishing my project presentation!	joy	0.84	excited happy, today excited, project presentation, finishing project, presentation	No
2026-03-03 13:49	Today I'm excited and happy after finishing my project presentation!	joy	0.84	excited happy, today excited, project presentation, finishing project, presentation	No

Figure 17 Mood History Page – Journal Records Table

The Mood History page shows a chronological log of all user journal entries. Each entry includes the date, written text, detected emotion, confidence score, extracted keywords, and whether the entry was anonymous. It helps users track emotional patterns over time and understand changes in their mental well-being. The page is accessible through the top navigation bar, along with links to the Dashboard, Reports, Admin panel, and Logout.

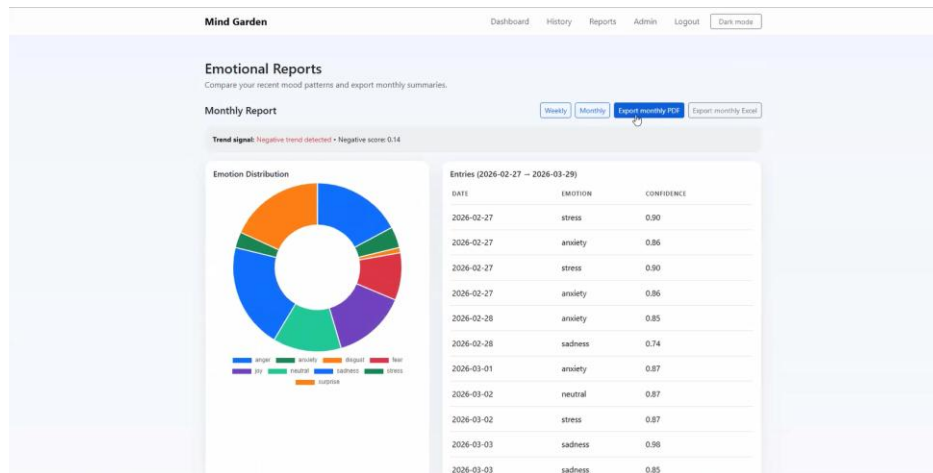


Figure 18 Emotional Reports Page – Monthly Report (Light Mode)

The Emotional Reports page provides mood analytics over a selected time period and allows users to export reports. It includes a trend indicator showing whether emotional patterns are improving or declining, such as a “Negative trend detected” alert. The page features a donut chart displaying emotion distribution and a table listing individual entries with date, emotion, and confidence scores.

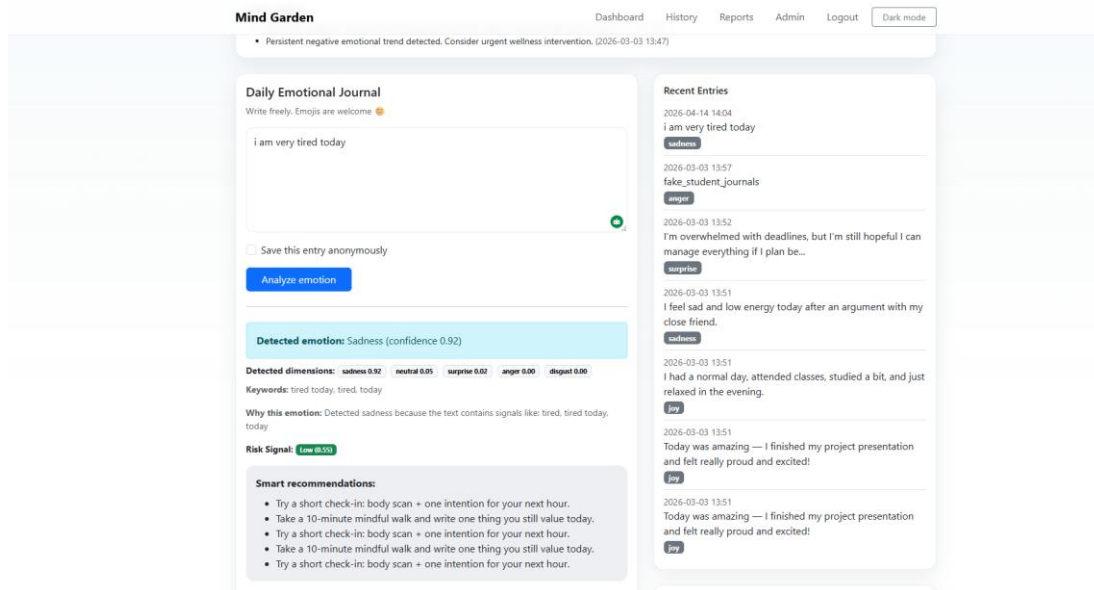


Figure 19 Dashboard – Daily Emotional Journal with NLP Analysis Result

The Dashboard is the main screen where users submit and analyze daily emotional journal entries. It includes a text input form with an option to submit anonymously and a button to analyze emotions using NLP. After submission, the system displays detected emotions with confidence scores, extracted keywords, and an explanation of the results, along with a risk level indicator. It also provides personalized wellness recommendations based on the detected mood.

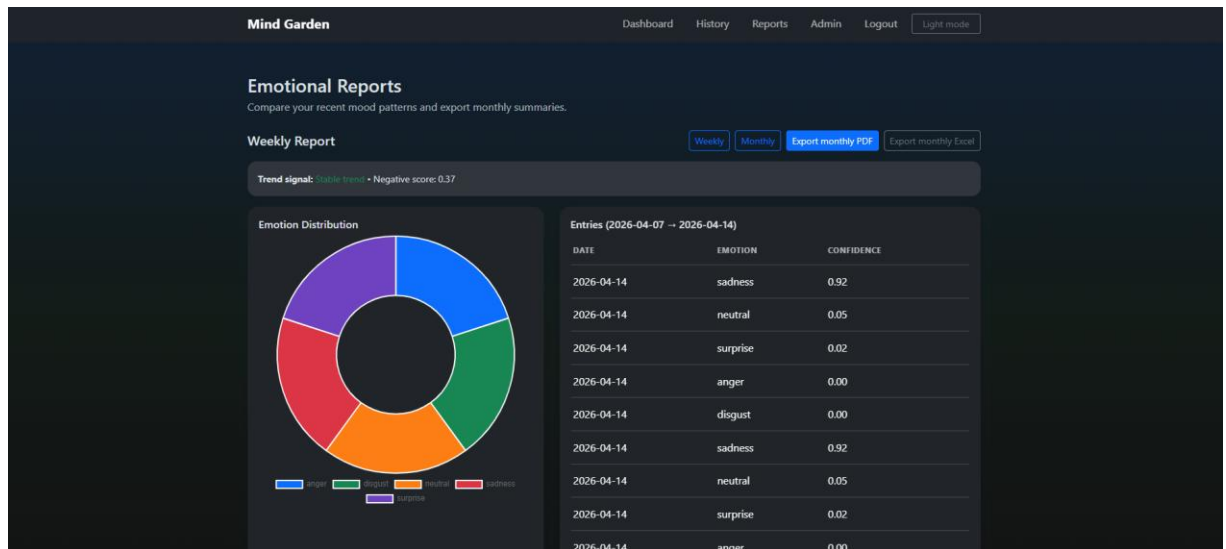


Figure 20 Emotional Reports Page – Weekly Report (Dark Mode)

This screenshot shows the Emotional Reports page in Dark Mode with the Weekly Report view enabled for the period 2026-04-07 to 2026-04-14. It includes a trend banner showing a “Stable trend” with a negative score of 0.37, indicating no major emotional decline. The page displays a donut chart for weekly emotion distribution and a table listing journal entries with date, emotion, and confidence scores. Users can switch between weekly and monthly views and export reports as PDF or Excel, with Dark Mode improving usability in low-light settings.

Create your account'."/>

Mind Garden

Login Register Dark mode

Logged out successfully.

Welcome back
Sign in to access your Mind Garden dashboard.

Email
admin@mindgarden.local

Password

Sign in

New here? [Create your account](#)

Figure 21 Login Page – User Authentication Screen

The Login page is a secure authentication screen for users and admins. It includes a simple form with email and password fields and a “Sign in” button. Users can access the registration page via a sign-up link, and a logout success message may appear when applicable. The page also supports Dark Mode and provides navigation links for Login and Register for unauthenticated users.

Sign in'."/>

Mind Garden

Login Register Dark mode

Create your account
Start your AI-assisted emotional wellness journey.

Full name
test

Email
admin@mindgarden.local

Password

Create account

Already registered? [Sign in](#)

Figure 22 Register Page – New Account Creation Screen

The Register page lets new users create an account for the Mind Garden platform. It includes a simple form with fields for full name, email, and password, along with a “Create account” button. Once submitted, the system validates the input, checks for

duplicate emails, securely hashes the password, and stores the new user in the database. It also provides a link for existing users to go back to the Login page and is accessible without authentication.

5.5.2 Admin interfaces

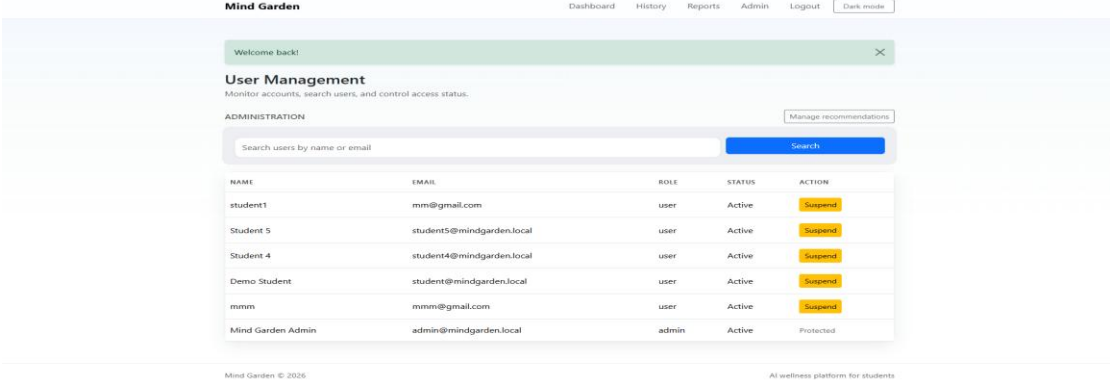


Figure 23User Management Dashboard

This interface allows administrators to monitor registered accounts, search users by name or email, and manage access status. The table displays user details including name, email, role, and account status, with action buttons for suspension or protection. The design emphasizes clarity and administrative control for secure platform management.

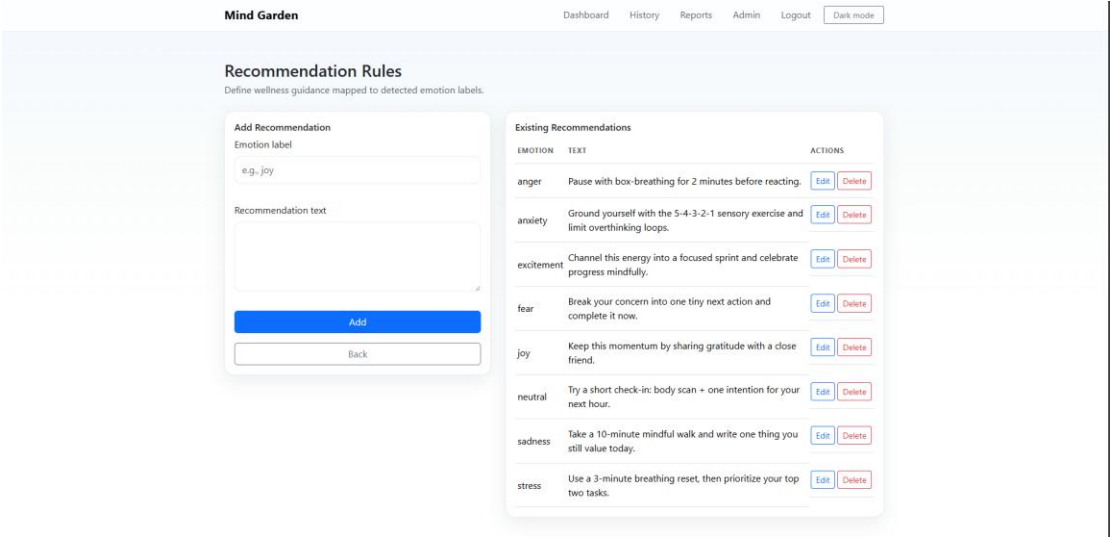


Figure 24 Recommendation Rules Panel

This panel defines personalized wellness recommendations mapped to detected emotions. Administrators can add new rules by specifying an emotion label and recommendation text. The table lists existing recommendations with options to edit or

delete, ensuring adaptability and continuous refinement of guidance. This feature highlights the system's AI-driven personalization and cultural sensitivity.

6. Testing, Analysis and Evaluation

6.1 Testing

To ensure the reliability and correctness of the Mind Garden system, several types of testing were conducted throughout the development process:

1. Functional Testing

Functional testing was performed to verify that all system features operate according to the specified requirements.

Tested functionalities include:

- User registration and login
- Writing and submitting emotional posts
- Emotion detection using NLP
- Displaying personalized recommendations
- Viewing mood history and reports
- Admin management (users & recommendations)

All major features were tested using different input scenarios, and the system behaved as expected.

2. Unit Testing

Individual components were tested independently to ensure correctness.

Examples:

- NLP emotion analysis function
- Text preprocessing module
- Recommendation generation logic

Each function returned expected outputs when tested with controlled inputs.

6.2 Analysis

The analysis phase focused on evaluating how effectively the system meets user needs and technical goals.

1. System Performance Analysis

The system demonstrated stable performance under normal usage conditions. NLP processing remained within acceptable response time limits, ensuring a smooth user experience.

2. Accuracy of Emotion Detection

The NLP model showed reasonable accuracy in detecting emotions from user text.

- High accuracy for clear emotional expressions
- Moderate accuracy for complex or mixed emotions

This indicates that while the model is effective, it may require further fine-tuning for edge cases.

3. User Experience Analysis

Based on initial feedback:

- Users found the interface simple and easy to use
- The anonymous feature increased comfort in sharing emotions
- Visual reports helped users understand emotional trends

4. System Limitations

- The NLP model is primarily optimized for English text
- Recommendations are rule-based (not fully adaptive yet)
- Limited real-world dataset for training and validation

6.3 Evaluation

The system was evaluated based on predefined criteria:

1. Functionality

The system successfully implements all core features:

- Emotion detection
- Personalized recommendations
- Mood tracking and reporting

2. Performance

- Fast response time (3–5 seconds)
- Smooth navigation and loading

3. Usability

- Clean and simple interface
- Easy navigation
- Suitable for non-technical users

4. Security & Privacy

- Secure authentication
- Encrypted passwords
- Anonymous usage supported

7. Issues

During development, several challenges were encountered:

1. NLP Model Limitations

- Difficulty in detecting mixed or subtle emotions
- Dependency on pre-trained models

2. Data Availability

- Lack of labeled datasets for emotional text
- Limited real user data for testing

3. Performance Constraints

- Transformer models require higher computational resources
- Slower performance on low-end devices

4. Integration Complexity

- Connecting NLP module with backend required careful handling
- Managing database relationships and consistency

5. Security Concerns

- Ensuring safe storage of sensitive psychological data
- Preventing unauthorized admin access

Table 12 Issues

No.	Issue	Solution / Workaround	Status
1	NLP model difficulty detecting mixed or subtle emotions	Used pre-trained transformer models with fine-tuning on emotional text datasets	Resolved
2	Lack of labeled datasets for emotional text	Relied on open-source datasets and public emotional text corpora for training	Partially Resolved
3	Transformer models require high computational resources; slower on low-end devices	Optimized model loading and implemented lazy initialization to reduce response time	Resolved
4	Connecting NLP module with Flask backend required careful handling; database consistency issues	Adopted service-layer architecture to separate NLP logic from routes and used SQLAlchemy ORM for consistent DB access	Resolved
5	Ensuring safe storage of sensitive psychological data; preventing unauthorized admin access	Applied bcrypt password hashing, session-based authentication, and role-based access control for admin routes	Resolved

8. Engineering Tools and Standards

1. Development Tools

- Programming Language: Python
- Framework: Flask
- Database: SQLite
- Frontend: HTML, CSS, JavaScript, Bootstrap

2. AI & Data Tools

- Transformers
- PyTorch
- Sentence Transformers

3. Development Environment

- VS Code
- Git (version control)

4. Engineering Standards

- Modular design (separation of concerns)
- MVC-like architecture

- Code reusability and maintainability
- Secure coding practices
- RESTful API design

9. Teamwork

The Mind Garden project was developed collaboratively by a team of students, where each member contributed to different aspects of the system.

Table 13 Teamwork

Team Member	Role	Responsibilities
Lamya Faisal Alahmari	Project Leader	Project planning, task distribution, overall supervision
Fajr Fahad Alshammri	Backend Developer	Authentication system, API development, database integration
May Mazah Aldafeeri	Frontend Developer	UI design, dashboard, user interaction
Dana Mohammed Al-Anzi	AI Developer	NLP model implementation, emotion analysis
Reham Mused	Database Developer	Database design, ER diagram, data management
Atheer Abdullah Alfaifi	System Analyst	Requirements analysis, use case diagrams
Aryam Ayed Al-Rashidi	Tester / QA	Testing, validation, performance checking
Bashayer Melfi Alrsheedi	Documentation Specialist	Report writing, formatting, diagrams

10. Progress

The following table presents the current progress of the Mind Garden project. Each task includes the responsible owner, description, time span, and status.

Phase 1: Requirements

Table 14 Phase 1: Requirements

Task	Owner	Description	Timespan	Status
Idea Proposal	All Team Members	Brainstorming and selecting the project idea (Mind Garden).	Week 1	Completed
Requirements Collection	All Team Members	Gathering functional and non-functional requirements.	Week 1 – Week 2	Completed
Requirements Analysis	System Analyst (Team)	Analyzing and refining system requirements.	Week 2	Completed
Project Plan	Project Leader	Creating project timeline and task allocation.	Week 2	Completed

Phase 2: System Design

Table 15 Phase 2: System Design

Task	Owner	Description	Timespan	Status
System Architecture Design	Backend Developer	Designing three-tier architecture and component structure.	Week 3	Completed
Use Case Diagram	UML Designer	Creating use case diagram for User and Admin.	Week 3	Completed
ER Diagram	Database Designer	Designing database schema and relationships.	Week 4	Completed
Sequence Diagram	UML Designer	Modeling system interaction flow.	Week 4	Completed
Prototype Development	Frontend + Backend	Creating clickable system prototype.	Week 5	Completed

Phase 3: Implementation

Table 16 Phase 3: Implementation

Task	Owner	Description	Timespan	Status
Database Implementation	Database Developer	Creating database tables and relationships.	Week 6 – Week 7	In Progress
NLP Model Development	AI Developer	Implementing preprocessing and emotion classification model	Week 7 – Week 9	In Progress
Backend Development	Backend Developer	Developing authentication, API endpoints, and logic.	Week 7 – Week 10	In Progress
Frontend Development	Frontend Developer	Implementing UI and dashboard.	Week 8 – Week 11	In Progress
Recommendation Engine	AI Developer	Implementing rule-based recommendation system.	Week 8 – Week 9	Waiting for NLP module completion

11. Conclusions

11.1 Conclusions

The Mind Garden project successfully achieved its goal of developing an intelligent web-based platform that supports the mental well-being of university students. The system integrates Artificial Intelligence and Natural Language Processing (NLP) techniques to analyze users' emotional states based on their daily written inputs. The application provides a safe and user-friendly environment where students can express their feelings, receive personalized recommendations, and track their emotional patterns over time through visual reports. Furthermore, the project highlights the importance of combining technology with mental health support, offering an accessible alternative for students who may hesitate to seek professional help. Overall, the Mind Garden system represents a meaningful and practical solution that addresses real-world challenges faced by university students, and it lays a strong foundation for future improvements and expansion.

11.2 Future Work

valuable enhancement is the integration of an AI-powered chatbot that can provide real-time emotional support and enable interactive conversations with users, making the platform more engaging and responsive. Furthermore, adding voice input support through speech-to-text technology would allow users to express their emotions verbally, offering a more flexible and natural way of interaction. To improve accessibility and user engagement, the development of a mobile application for Android and iOS platforms is also recommended, enabling users to access the system anytime and anywhere. These enhancements, when implemented, will significantly strengthen the system's capabilities, making it more intelligent, user-friendly, and impactful in supporting students' mental health.

Reference

- [1] Sommerville, I. (2016). *Software Engineering* (10th ed.). Pearson Education.
- [2] Pallets Projects. (2023). *Flask Documentation (Version 2.x)*. Retrieved from <https://flask.palletsprojects.com/>
- [3] Lucidchart, "Uml sequence diagram," [Online]. Available: <https://www.lucidchart.com/pages/uml-sequence-diagram>.
- [4] Jurafsky, D., & Martin, J. H. (2023). *Speech and Language Processing* (3rd ed. draft). Stanford University. <https://web.stanford.edu/~jurafsky/slp3/>
- [5] Bird, S., Klein, E., & Loper, E. (2009). *Natural Language Processing with Python*. O'Reilly Media.
- [6] Boehm, B. (1988). A spiral model of software development and enhancement. *Computer*, 21(5), 61–72. <https://doi.org/10.1109/2.59>
- [7] World Health Organization (WHO). (2023). *Mental health of adolescents and young adults*. Retrieved from: <https://www.who.int/news-room/fact-sheets/detail/adolescent-mental-health>

Appendix

1) Project questionnaire

تطبيق ويب ذكي لدعم الصحة النفسية لطلاب كلية علوم وهندسة الحاسب الآلي (Mind (garden

هذا الاستبيان جزء من مشروع تخرج في كلية علوم وهندسة الحاسب الآلي، ويهدف إلى تطوير تطبيق ويب ذكي يساعد طلاب الجامعة على متابعة صحتهم النفسية وتقديم نصائح مناسبة لهم باستخدام تقنيات الذكاء الاصطناعي.

This questionnaire is part of a graduation project in the College of Computer Science and Engineering. It aims to develop an intelligent web application that helps university students monitor their mental health and receive appropriate advice using artificial intelligence technologies.

الاستبيان مجهول الهوية بالكامل ولن يتم جمع أي معلومات شخصية، والإجابات ستستخدم لأغراض أكاديمية فقط.

This questionnaire is completely anonymous, and no personal information will be collected. All responses will be used for academic purposes only.

الوقت المتوقع للإجابة: دقيقة إلى دقيقتين.

.Estimated time to complete: 1-2 minutes

engmohamedsw@gmail.com [Switch account](#)

Not shared

اللغة العربية

Age Group

- ☐ أقل من 18 سنة
- ☐ 18 - 20 سنة
- ☐ 21 - 23 سنة
- ☐ 24 - 26 سنة
- ☐ أكثر من 26 سنة

كم ساعة تقضي يومياً على الأجهزة (موبايل/كمبيوتر)؟
How many hours do you spend daily on devices (mobile/computer)

- ☐ أقل من ساعتين
- ☐ 2 - 4 ساعات
- ☐ 4 - 7 ساعات
- ☐ أكثر من 7 ساعات

هل تشعر بضغط دراسي خلال الفصل الدراسي؟
Do you feel academic stress during the semester

- ☐ نعم
- ☐ لا
- ☐ ربما

ما أكثر سبب يسبب لك التوتر؟
What is the main cause of your stress

- ☐ كثرة الواجبات
- ☐ الامتحانات
- ☐ المشاريع
- ☐ ضيق الوقت
- ☐ السهر و قلة النوم
- ☐ Other: _____

عندما تشعر بالضغط النفسي ماذا تفعل عادة؟
What do you usually do when you feel psychological stress

- ☐ أجاهله
- ☐ أتحدث مع صديق
- ☐ أستخدم وسائل التواصل
- ☐ أكتب مشاعري
- ☐ لا أفعل شيء

هل سبق لك استخدام تطبيق للصحة النفسية أو تتبع المزاج؟
Have you ever used a mental health or mood tracking app

- ☐ نعم
- ☐ لا

لماذا قد تردد في الذهاب إلى معالج نفسي؟
Why might you hesitate to visit a therapist

- ☐ لا أرى فائدة واضحة من العلاج النفسي
- ☐ أظنها مشكلة بسيطة
- ☐ أفضل حل مشاكلي بنفسى
- ☐ التكلفة أو المواقيد

هل تحب وجود تطبيق تستطيع فيه كتابة مشاعرك يوميًا ويحلل حالتك النفسية تلقائيًا؟
Would you like an app where you can write your feelings daily and have your mental state analyzed automatically

- ☐ نعم
- ☐ لا
- ☐ ربما

هل تفضل أن تكون النصائح مخصصة لك بناءً على حالتك النفسية؟
Do you prefer personalized advice based on your mental state

- ☐ نعم
- ☐ لا
- ☐ ربما

هل تشعر بالراحة في استخدام تطبيق مجهول الهوية (لا يظهر اسمك) لمشاركة مشاعرك؟
Do you feel comfortable using an anonymous app (that does not show your name) to share your feelings

- ☐ نعم
- ☐ لا
- ☐ ربما

هل تعتقد أن تطبيق كهذا قد يساعد في تحسين حالتك النفسية أو تنظيم دراستك؟
Do you think such an app could help improve your mental state or organize your studies

- ☐ نعم
- ☐ لا
- ☐ ربما

Figure 25 Project questionnaire

B. Source Code

The complete source code is available in the project repository. Below are the main source files of the Mind Garden web application.

B.1 requirements.txt

This file lists all Python dependencies required to run the Mind Garden application:

```
Flask==3.1.0
Flask-Login==0.6.3
Flask-SQLAlchemy==3.1.1
Flask-Migrate==4.1.0
bcrypt==4.2.1
transformers==4.48.0
torch==2.6.0
sentence-transformers==3.4.1
keybert==0.8.5
pandas==2.2.3
openpyxl==3.1.5
reportlab==4.2.5
python-dotenv==1.0.1
```

B.2 run.py – Application Entry Point

This is the main entry point for starting the Flask application:

```
from app import create_app

# Entry point for local development and production WSGI servers.
app = create_app()

if __name__ == "__main__":
    app.run(debug=True)
```

B.3 config.py – Application Configuration

This file defines all application configuration settings including database URI, secret keys, and NLP model parameters:

```
import os
from datetime import timedelta

class Config:
    # Core Flask settings
    SECRET_KEY = os.getenv("SECRET_KEY", "dev-secret-change-me")
    SQLALCHEMY_DATABASE_URI = os.getenv("DATABASE_URL",
    "sqlite:///mind_garden.db")
    SQLALCHEMY_TRACK_MODIFICATIONS = False
    SQLALCHEMY_ENGINE_OPTIONS = {
        "pool_pre_ping": True,
        "connect_args": {"timeout": 30},
    }

    # Session and auth settings
    REMEMBER_COOKIE_DURATION = timedelta(days=14)

    # NLP model configuration
    HF_EMOTION_MODEL = os.getenv(
```

```

        "HF_EMOTION_MODEL", "j-hartmann/emotion-english-distilroberta-
base"
    )
    HF_RISK_MODEL = os.getenv("HF_RISK_MODEL", "facebook/bart-large-
mnli")
    USE_HF_RISK_MODEL = os.getenv("USE_HF_RISK_MODEL", "0") == "1"

    # App-level defaults
    POSTS_PER_PAGE = 20

```

B.4 app/__init__.py – Application Factory

This file contains the Flask application factory function that initializes all extensions, registers blueprints, and seeds the database:

```

from flask import Flask
import bcrypt
from config import Config
from app.extensions import db, login_manager, migrate
from app.models import EmotionResult, EmotionalPost, Recommendation, User

def create_app(config_class=Config):
    app = Flask(__name__, instance_relative_config=True)
    app.config.from_object(config_class)

    # Initialize Flask extensions
    db.init_app(app)
    login_manager.init_app(app)
    migrate.init_app(app, db)
    login_manager.login_view = "auth.login"

    # Register blueprints
    from app.auth.routes import auth_bp
    from app.main.routes import main_bp
    from app.admin.routes import admin_bp
    from app.api.routes import api_bp

    app.register_blueprint(auth_bp)
    app.register_blueprint(main_bp)
    app.register_blueprint(admin_bp, url_prefix="/admin")
    app.register_blueprint(api_bp, url_prefix="/api/v1")

    with app.app_context():
        db.create_all()
        seed_default_data()

    return app

```

B.5 app/auth/routes.py – Authentication Routes

This module handles user registration, login, and logout functionality using Flask-Login and bcrypt password hashing:

```

from __future__ import annotations

import bcrypt
from flask import Blueprint, flash, redirect, render_template, request,
url_for
from flask_login import current_user, login_required, login_user,
logout_user

```

```

from app.extensions import db
from app.models import User

auth_bp = Blueprint("auth", __name__, template_folder="../templates")

@auth_bp.route("/register", methods=["GET", "POST"])
def register():
    if current_user.is_authenticated:
        return redirect(url_for("main.dashboard"))

    if request.method == "POST":
        name = request.form.get("name", "").strip()
        email = request.form.get("email", "").strip().lower()
        password = request.form.get("password", "")

        if not name or not email or not password:
            flash("Please fill in all fields.", "danger")
            return render_template("auth/register.html")

        if User.query.filter_by(email=email).first():
            flash("Email is already registered.", "warning")
            return render_template("auth/register.html")

        hashed = bcrypt.hashpw(password.encode("utf-8"),
                                bcrypt.gensalt()).decode("utf-8")
        user = User(name=name, email=email, password=hashed, role="user")
        db.session.add(user)
        db.session.commit()

        flash("Registration successful. Please login.", "success")
        return redirect(url_for("auth.login"))

    return render_template("auth/register.html")

@auth_bp.route("/login", methods=["GET", "POST"])
def login():
    if current_user.is_authenticated:
        return redirect(url_for("main.dashboard"))

    if request.method == "POST":
        email = request.form.get("email", "").strip().lower()
        password = request.form.get("password", "")

        user = User.query.filter_by(email=email).first()
        if not user:
            flash("Invalid credentials.", "danger")
            return render_template("auth/login.html")

        if not user.is_active:
            flash("Your account has been deactivated. Contact support.",
                  "danger")
            return render_template("auth/login.html")

        valid = bcrypt.checkpw(password.encode("utf-8"),
                                user.password.encode("utf-8"))
        if not valid:
            flash("Invalid credentials.", "danger")
            return render_template("auth/login.html")

```

```

        login_user(user, remember=True)
        flash("Welcome back!", "success")

        if user.role == "admin":
            return redirect(url_for("admin.users"))
            return redirect(url_for("main.dashboard"))

    return render_template("auth/login.html")

@auth_bp.route("/logout")
@login_required
def logout():
    logout_user()
    flash("Logged out successfully.", "info")
    return redirect(url_for("auth.login"))

```

B.6 app/main/routes.py – Main Application Routes

This module handles the core user-facing functionality including the dashboard, emotional post submission, history view, and report generation:

```

from __future__ import annotations

from collections import Counter
from datetime import datetime, timedelta

from flask import Blueprint, Response, flash, redirect, render_template,
request, url_for
from flask_login import current_user, login_required
from sqlalchemy.exc import OperationalError

from app.extensions import db
from app.models import EmotionalPost, EmotionResult,
RecommendationHistory
from app.services.emotion_service import analyze_post,
ensure_post_embeddings, to_embedding_json
from app.services.recommendation_service import generate_recommendations
from app.services.trend_service import compute_user_trend

main_bp = Blueprint("main", __name__, template_folder="../templates")

@main_bp.route("/")
def index():
    if current_user.is_authenticated:
        return redirect(url_for("main.dashboard"))
    return render_template("main/index.html")

@main_bp.route("/dashboard", methods=["GET", "POST"])
@login_required
def dashboard():
    detected = None
    recommendation_items = []

    if request.method == "POST":
        text = request.form.get("text", "").strip()
        is_anonymous = request.form.get("is_anonymous") == "on"

        if not text:

```

```

        flash("Please write something before submitting.", "warning")
        return redirect(url_for("main.dashboard"))

    analysis = analyze_post(text)
    post = EmotionalPost(
        user_id=current_user.id,
        text=text,
        is_anonymous=is_anonymous,
        keywords=", ".join(analysis["keywords"]),
        embedding_vector=to_embedding_json(analysis["embedding"]),
    )
    db.session.add(post)
    db.session.flush()

    for emotion in analysis["emotions"]:
        db.session.add(
            EmotionResult(
                post_id=post.id,
                emotion_label=str(emotion["label"]).lower(),
                confidence_score=float(emotion["score"]),
            )
        )

    recommendation_items = generate_recommendations(
        user_id=current_user.id,
        post=post,
        emotions=analysis["emotions"],
        embedding=analysis["embedding"],
    )
    db.session.commit()

    detected = {
        "label": analysis["top_emotion"],
        "confidence": analysis["top_confidence"],
    }

    posts = (
        EmotionalPost.query.filter_by(user_id=current_user.id)
        .order_by(EmotionalPost.created_at.desc())
        .limit(7)
        .all()
    )

    return render_template(
        "main/dashboard.html",
        posts=posts,
        detected=detected,
        recommendation_items=recommendation_items,
    )

@main_bp.route("/history")
@login_required
def history():
    posts = (
        EmotionalPost.query.filter_by(user_id=current_user.id)
        .order_by(EmotionalPost.created_at.desc())
        .all()
    )
    return render_template("main/history.html", posts=posts)

@main_bp.route("/reports")
@login_required

```

```

def reports():
    period = request.args.get("period", "weekly")
    days = 7 if period == "weekly" else 30
    end_date = datetime.utcnow()
    start_date = end_date - timedelta(days=days)

    rows = (
        db.session.query(
            EmotionResult.emotion_label,
            EmotionResult.confidence_score,
            EmotionalPost.created_at
        )
        .join(EmotionalPost, EmotionResult.post_id == EmotionalPost.id)
        .filter(EmotionalPost.user_id == current_user.id)
        .filter(EmotionalPost.created_at >= start_date)
        .order_by(EmotionalPost.created_at.asc())
        .all()
    )

    emotion_counts = Counter(label for label, _, _ in rows)
    trend = compute_user_trend(current_user.id, days=days)

    return render_template(
        "main/reports.html",
        period=period,
        emotion_counts=emotion_counts,
        trend=trend,
    )

```

B.7 app/admin/routes.py – Admin Panel Routes

This module handles all administrative functionality including user management and wellness recommendation management:

```

from __future__ import annotations

from functools import wraps

from flask import Blueprint, flash, redirect, render_template, request, url_for
from flask_login import current_user, login_required

from app.extensions import db
from app.models import Recommendation, User

admin_bp = Blueprint("admin", __name__, template_folder="../templates")

def admin_required(view):
    @wraps(view)
    @login_required
    def wrapped(*args, **kwargs):
        if current_user.role != "admin":
            flash("Admin access required.", "danger")
            return redirect(url_for("main.dashboard"))
        return view(*args, **kwargs)
    return wrapped

@admin_bp.route("/users")
@admin_required
def users():

```

```

q = request.args.get("q", "").strip().lower()
query = User.query
if q:
    query = query.filter(
        (User.name.ilike(f"%{q}%")) | (User.email.ilike(f"%{q}%"))
    )
all_users = query.order_by(User.created_at.desc()).all()
return render_template("admin/users.html", users=all_users, q=q)

@admin_bp.route("/users/<int:user_id>/toggle", methods=["POST"])
@admin_required
def toggle_user_status(user_id: int):
    user = User.query.get_or_404(user_id)
    if user.role == "admin":
        flash("Admin accounts cannot be deactivated.", "warning")
        return redirect(url_for("admin.users"))
    user.is_active = not user.is_active
    db.session.commit()
    flash("User status updated.", "success")
    return redirect(url_for("admin.users"))

@admin_bp.route("/recommendations", methods=["GET", "POST"])
@admin_required
def recommendations():
    if request.method == "POST":
        emotion_label = request.form.get("emotion_label",
            "").strip().lower()
        recommendation_text = request.form.get("recommendation_text",
            "").strip()
        if not emotion_label or not recommendation_text:
            flash("Both fields are required.", "danger")
            return redirect(url_for("admin.recommendations"))
        db.session.add(
            Recommendation(
                emotion_label=emotion_label,
                recommendation_text=recommendation_text
            )
        )
        db.session.commit()
        flash("Recommendation added.", "success")
        return redirect(url_for("admin.recommendations"))

    rows =
Recommendation.query.order_by(Recommendation.emotion_label.asc()).all()
    return render_template("admin/recommendations.html",
        recommendations=rows)

@admin_bp.route("/recommendations/<int:recommendation_id>/edit",
    methods=["GET", "POST"])
@admin_required
def edit_recommendation(recommendation_id: int):
    row = Recommendation.query.get_or_404(recommendation_id)
    if request.method == "POST":
        row.emotion_label = request.form.get("emotion_label",
            "").strip().lower()
        row.recommendation_text = request.form.get("recommendation_text",
            "").strip()
        db.session.commit()
        flash("Recommendation updated.", "success")
        return redirect(url_for("admin.recommendations"))
    return render_template("admin/edit_recommendation.html", item=row)

```



```

@admin_bp.route("/recommendations/<int:recommendation_id>/delete",
methods=["POST"])
@admin_required
def delete_recommendation(recommendation_id: int):
    row = Recommendation.query.get_or_404(recommendation_id)
    db.session.delete(row)
    db.session.commit()
    flash("Recommendation deleted.", "info")
    return redirect(url_for("admin.recommendations"))

```

B.8 app/utils/nlp_service.py – NLP Emotion Analysis Service

This module implements the core NLP-based emotion analysis using the HuggingFace Transformers library with a graceful fallback mechanism:

```

from __future__ import annotations

from dataclasses import dataclass
from typing import Dict, Optional

from flask import current_app
from app.utils.text_processing import clean_text, simple_tokenize

@dataclass
class EmotionAnalysis:
    label: str
    confidence: float
    cleaned_text: str
    token_count: int

    _classifier = None
    _tokenizer = None
    _load_error: Optional[str] = None

    def _load_pipeline() -> None:
        """Lazy-load HuggingFace pipeline and tokenizer once per process."""
        global _classifier, _tokenizer, _load_error
        if _classifier is not None or _load_error is not None:
            return

        try:
            from transformers import AutoTokenizer, pipeline

            model_name = current_app.config["HF_EMOTION_MODEL"]
            _tokenizer = AutoTokenizer.from_pretrained(model_name)
            _classifier = pipeline("text-classification", model=model_name,
top_k=1)
        except Exception as exc:
            _load_error = str(exc)

    def analyze_emotion(text: str) -> Dict[str, float | str | int]:
        """
        Analyze emotion for a journal entry.
        Returns emotion label + confidence and processing metadata.
        """
        cleaned = clean_text(text)

```

```

tokens = simple_tokenize(cleaned)

_load_pipeline()

if _classifier is not None and _tokenizer is not None:
    _tokenizer(cleaned, truncation=True, max_length=256)
    raw = _classifier(cleaned)
    item = raw[0][0] if raw and isinstance(raw[0], list) else raw[0]
    label = str(item.get("label", "neutral")).lower()
    score = float(item.get("score", 0.0))
else:
    # Graceful fallback when model is unavailable
    text_l = cleaned.lower()
    fallback = {
        "joy": ["happy", "grateful", "excited", "great", "good"],
        "sadness": ["sad", "down", "depressed", "lonely"],
        "anger": ["angry", "mad", "frustrated", "annoyed"],
        "fear": ["anxious", "scared", "worried", "nervous"],
    }
    label = "neutral"
    score = 0.55
    for emotion, keywords in fallback.items():
        if any(keyword in text_l for keyword in keywords):
            label = emotion
            score = 0.7
            break

return {
    "label": label,
    "confidence": round(score, 4),
    "cleaned_text": cleaned,
    "token_count": len(tokens),
    "load_error": _load_error,
}

```

B.9 app/utls/text_processing.py – Text Processing Utilities

This module provides text cleaning and tokenization utilities used by the NLP pipeline:

```

import re
from typing import List

def clean_text(text: str) -> str:
    """Clean input text while preserving emotional content and emojis."""
    text = text.strip()
    text = re.sub(r"https?://\S+|www\.\S+", "", text)
    text = re.sub(r"\s+", " ", text)
    return text

def simple_tokenize(text: str) -> List[str]:
    """Basic tokenizer used for telemetry and debugging NLP behavior."""
    return re.findall(r"[\w']+", text.lower())

```